ASSIGNMENT 4 – CO4

TASK 3: FROM SKETCH TO LAYOUT

Campus Library Search Screen

Course: Programming in Java (CSA09)
Course Outcome: CO4 – Event-driven GUI design
Task Weightage: 9 Marks

Layout selection, nested container design, AWT implementation, resizing verification and alternative layout analysis.

1. Requirement Analysis

The screen must contain a title at the top; a search row with a text field, category Choice and Search button; a results area in the centre; Prev, Next and Reserve buttons at the bottom; and a status message. On resizing, the results area must grow or shrink while the search row and bottom controls keep their preferred size.

The implementation uses Java AWT, including Frame, Panel, TextField, Choice, Button, List and Label. Sample book data is included so that search, paging and reserve feedback can be demonstrated without requiring a database; the task does not specify database integration.

2. Task 3.1 – Choose the Layout Managers

Container / region	Layout manager	Reason
Main Frame	BorderLayout	Divides the window into NORTH, CENTER and SOUTH. CENTER receives remaining space when the frame is resized.
Top panel	BorderLayout	Places the title at NORTH and the search row at SOUTH without making them compete with the results area.
Search row panel	FlowLayout	Keeps the label, text field, category Choice and Search button grouped at their preferred sizes.
Bottom panel	BorderLayout	Places navigation controls at NORTH and status message at SOUTH.
Navigation panel	FlowLayout	Keeps Prev, Next and Reserve buttons in a single compact row at preferred sizes.
Results area	AWT List in CENTER	Expands to use the available central space as the window grows.

GridLayout is not selected because it gives equal-sized cells and may stretch the search controls or buttons. CardLayout is intended for switching between stacked screens, which is not required here.

3. Task 3.2 – Nested Container Structure

```
Frame: LibrarySearch [BorderLayout]
|
+-- Panel: topPanel [BorderLayout] ---------------- NORTH
|   +-- Label: title [NORTH]
|   +-- Panel: searchPanel [FlowLayout] [SOUTH]
|       +-- Label: Search
|       +-- TextField: searchField
|       +-- Label: Category
|       +-- Choice: categoryChoice
|       +-- Button: searchButton
|
+-- List: resultsList ---------------------------- CENTER
|
+-- Panel: bottomPanel [BorderLayout] ------------- SOUTH
    +-- Panel: navigationPanel [FlowLayout] [NORTH]
    |   +-- Button: Prev
    |   +-- Button: Next
    |   +-- Button: Reserve
    +-- Label: statusLabel [SOUTH]
```

4. Task 3.3 – Implementation

The supplied LibrarySearch.java implements the required screen using AWT. It includes a sample catalogue of 16 books, category-based search, page navigation, reserve feedback and a window-closing listener.

Event handling: Search button and Enter in the search field call performSearch(). Prev and Next update currentPage and refresh the visible results. Reserve checks whether a row is selected and displays a status message. WindowAdapter handles the close event and disposes the frame.

Search behaviour: The category Choice supports All, Title, Author and Category. The results are displayed eight per page. This sample application demonstrates the layout and interactions; it does not connect to a live library database or create a real reservation.

5. Task 3.4 – Test Resizing Behaviour

Run the Java program and capture three real screenshots at small, medium and large window sizes. The screenshots should show the full window and demonstrate that the central List changes size while the title/search row and navigation/status

region remain in their assigned positions.

Screenshot	Test action	Expected observation
Small	Resize frame smaller	Results area becomes shorter/narrower; top and bottom controls remain grouped.
Medium	Use initial 760 × 500 size	All regions are visible and the results list has usable space.
Large	Maximize/enlarge frame	Results list receives most of the additional space; search and button rows retain preferred sizing.

Evidence note: The report does not include fabricated execution screenshots. Add your own screenshots after running the program and save them as output_small.png, output_medium.png and output_large.png in the Task03 folder.

6. Alternative Concept and Complexity Analysis

Alternative A – GridBagLayout: the same components can be placed using GridBagConstraints. The results component can receive weightx/weighty and fill BOTH, while search controls and navigation buttons can use preferred sizing. This offers precise, flexible constraints but requires more configuration and careful constraint management.

Alternative B – Absolute positioning: setLayout(null) and setBounds(x, y, width, height) manually position every component. A resize listener would be needed to recompute bounds. Without that extra logic, controls remain at old coordinates, overlap or become clipped as the window changes size.

Aspect	Nested BorderLayout + FlowLayout	GridBagLayout	Absolute positioning
Resizing	Centre expands naturally; fixed rows retain preferred sizing.	Can resize correctly when weights/fill/anchors are configured.	Does not adapt automatically; bounds must be recalculated.
Layout calculation	Simple region placement; generally O(n) in component count n.	Constraint processing; generally scales with number of components, with larger constant/configuration overhead.	Direct coordinate assignment is O(n) if all n components are repositioned.
Container objects	A small number of nested panels.	Can use fewer panels, but constraints are more complex.	May use fewer panels, but requires manual resize logic.
Maintainability	Clear and easy to adjust for this screen.	Flexible for complex forms; more verbose.	Fragile across screen sizes and font/display scaling.

Space/container analysis: Let n be the number of components and p the number of nested panels. The proposed design uses O(n + p) component/container references, with a small constant p. GridBagLayout can reduce panel nesting but stores layout constraints for components, also O(n). Absolute positioning can use fewer layout containers, but still stores component references and bounds (O(n)); the apparent container saving is offset by custom resize code. Exact runtime depends on the AWT implementation and platform, so Big-O should be treated as an order-of-growth comparison, not a measured benchmark.

7. Conclusion

Nested BorderLayout and FlowLayout provide the clearest solution for the stated resizing requirement. The central results List receives the extra space, while the search and navigation controls remain grouped at their preferred sizes. The AWT program demonstrates the required interactions and provides a practical basis for the three resizing screenshots.

8. Execution Procedure

```
Open Task03 in VS Code terminal:
    javac LibrarySearch.java
    java LibrarySearch

Capture screenshots at small, medium and large sizes.
Save them in Task03 as:
    output_small.png
    output_medium.png
    output_large.png
```